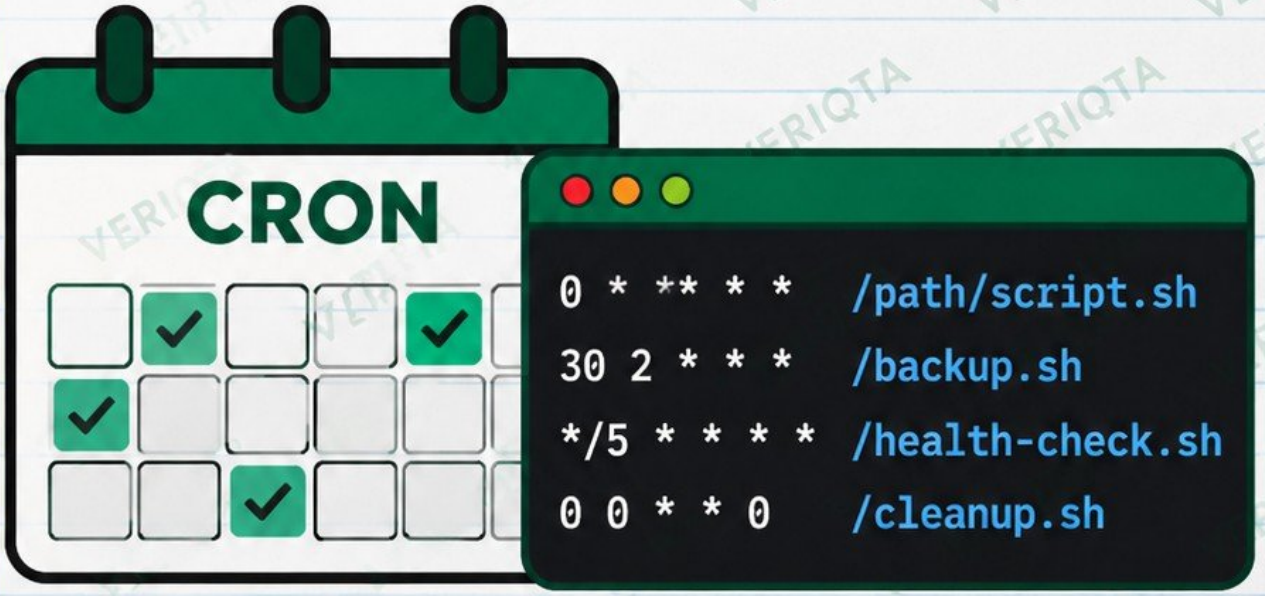
VERIQTA

The Cron Job Cookbook

30 Recipes You'll Actually Use



PRACTICAL RECIPES | REAL-WORLD EXAMPLES | TROUBLESHOOTING GUIDE | PRODUCTION BEST PRACTICES

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

Page 1: Cron Is Your Linux Automation Clock

VERIQTA

1. What is Cron?

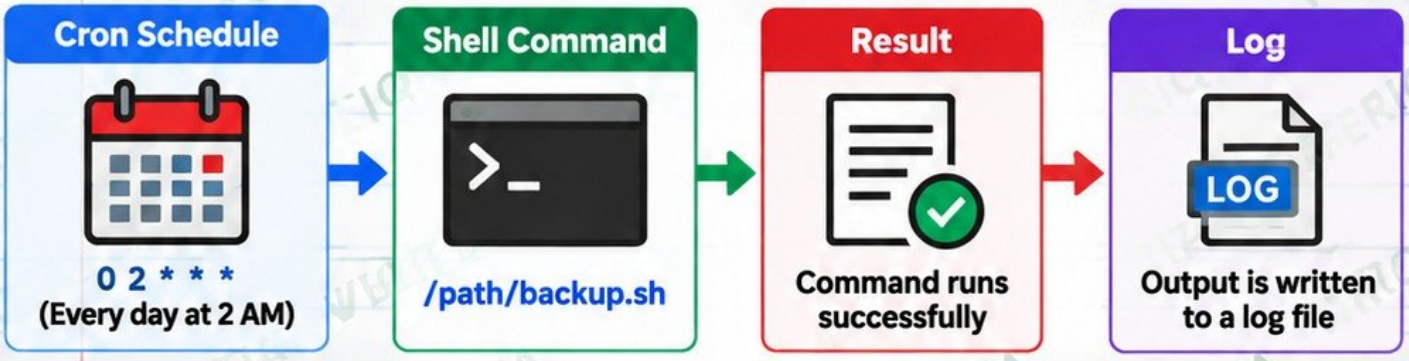
- Cron is a built-in Linux service (daemon) that runs scheduled commands automatically.
- It runs in the background and does not need a user to be logged in.
- You define what to run and when to run it using a crontab file.

2. Why Linux Servers Need It

- Automate backups.
- Run maintenance tasks.
- Clean up logs and temporary files.
- Monitor system health.
- Run application scripts.
- Keep services running smoothly.
- Reduce manual work and human error.

3. How Cron Works

1. You add a schedule and a command to a crontab file.
2. The cron daemon checks the schedule every minute.
3. When the time matches, it runs the command as the specified user.
4. The result is written to a log or output file.



4. Where Cron Is Configured

- Edit your user crontab: `crontab -e`
- List your crontab: `crontab -l`
- System-wide cron files: `/etc/cron.d/` `/etc/crontab`

5. Simple Example

This cron job writes the current date and time to a log every day at 2 AM.

```
0 2 * * *  echo "Backup run: $(date)" >> /var/log/backup.log
```

6. Key Takeaways

- Cron runs commands automatically on a schedule.
- It works even if no user is logged in.
- It is widely used for backups, maintenance, monitoring, and automation.
- Always log the output so you can verify the results.

Page 2: Read a Cron Schedule Without Guessing

VERIQTA

1. The Five Cron Time Fields

Minute	Hour	Day of Month	Month	Day of Week
0 - 59	0 - 23	1 - 31	1 - 12	0 - 7 (0 = Sun or 7 = Sun)

2. What Do the Symbols Mean?

*	Any value (every minute, hour, etc.)
,	Multiple values (e.g. 1,15,30)
-	Range of values (e.g. 1-5)
/	Step values (e.g. */5 = every 5)

Example: `0 */2 * * *` → Every 2 hours at minute 0

3. How to Read a Cron Schedule

`5 0 * * 1`

- **5** → Minute: 5
- **0** → Hour: 0 (midnight)
- ***** → Day of Month: Every day
- ***** → Month: Every month
- **1** → Day of Week: Monday

This schedule means:

- At 00:05 (12:05 AM)
- Every day of the month
- Every month
- Only on Monday

4. Important: Day-of-Month vs Day-of-Week Matching

- If both Day of Month and Day of Week are restricted (not *), the job runs when **EITHER** field matches (not both).
- This means: `0 0 1 * 1` runs on the 1st of every month **AND** every Monday (not only when the 1st is a Monday).

5. Common Schedule Examples

Schedule	Meaning	Example Use Case
* * * * *	Every minute	Run a quick health check
0 * * * *	Every hour at minute 0	Check disk usage hourly
0 0 * * *	Every day at midnight	Daily backup
5 0 * * 1	Every Monday at 00:05	Weekly report
0 0 1 * *	On the 1st of every month	Monthly billing report
0 18 * * 1-5	Weekdays at 6:00 PM	Run cleanup script on weekdays
*/15 * * * *	Every 15 minutes	Monitor application status
0 2 */7 * *	Every 7 days at 2:00 AM	Rotate logs weekly

Page 3: Your First Cron Job, Built Safely

VERIQTA

1. What We Will Do

- Create two real cron jobs:
  1. Write a timestamp to a test log every minute.
  2. Run a simple health-check script every five minutes.
- Learn how to use crontab -e and crontab -l.
- Set correct script permissions.
- Use absolute paths.
- Verify that the jobs actually run.

2. Cron Workflow



3. Manage Your Crontab

- Open your crontab for editing:

```
crontab -e
# Adds, updates or removes cron jobs
```

- List your existing cron jobs:

```
crontab -l
# Shows your current cron jobs
```

4. Recipe 01: Write a Timestamp Every Minute

- Add this line to your crontab:

```
* * * * * echo "$(date '+%Y-%m-%d %H:%M:%S')" \
        >> /home/ubuntu/cron-test.log
```

- Runs every minute.
- Appends the current date and time to a log file.
- Uses an absolute path.

5. Recipe 02: Run a Health-Check Script

- Create a simple health-check script:

```
sudo nano /usr/local/bin/health-check.sh
#!/bin/bash
echo "$(date '+%Y-%m-%d %H:%M:%S') - Service OK"
>> /var/log/health-check.log
```

- Make it executable:

```
sudo chmod +x /usr/local/bin/health-check.sh
```

- Add this line to your crontab:

```
*/5 * * * * /usr/local/bin/health-check.sh
```

- Runs every 5 minutes.
- Writes a timestamped health check message to a log file.

6. Check That Your Cron Jobs Are Running

- View the log for Recipe 01:

```
tail -n 5 /home/ubuntu/cron-test.log
```

- View the log for Recipe 02:

```
tail -n 5 /var/log/health-check.log
```

- Check the last time cron ran a job:

```
grep CRON /var/log/syslog | tail -n 5
```

7. Important Tips

- Always use absolute paths.
- Ensure scripts have execute permission.
- Test your script manually before scheduling.
- Check logs to confirm the job ran.
- If it doesn't run, check the cron logs and file permissions.

8. Expected Output Example

```
2026-10-24 14:01:00
2026-10-24 14:02:00
2026-10-24 14:03:00
2026-10-24 14:04:00
2026-10-24 14:05:00
```


@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

Page 4: Cron Environments Are Not Your Terminal

VERIQTA

1. Cron Runs With a Minimal Environment

- Cron runs with a limited set of environment variables such as: HOME, LOGNAME, PATH, SHELL.
- It does not load your shell profile files (.bashrc, .bash_profile, etc.).
- Your PATH may be different, so commands that work in your terminal may not be found.
- Always use absolute paths or set the PATH explicitly in the crontab or script.

2. Why Commands Work in Terminal but Fail in Cron

Interactive Terminal	Cron Environment
✓ Loads your shell files (e.g. .bashrc)	✗ Does not load shell files
✓ Has a full PATH	✗ Has a minimal PATH
✓ Has custom environment variables	✗ May not have your environment variables
✓ Commands are usually found and run successfully	✗ Commands may fail with command not found or other errors

3. Check the Cron Environment

- View the default environment used by cron:

```
env > /tmp/cron-env.log
```

- Check what PATH is available:

```
echo $PATH > /tmp/cron-path.log
```

- Compare with your terminal environment:

```
echo $PATH
```

4. Recipe 03: Run a Python Script Safely

1. Create a simple Python script.

```
sudo nano /opt/scripts/hello.py
#!/usr/bin/env python3
import datetime
print(f"Cron run at: {datetime.datetime.now()}")
```

2. Make it executable:

```
sudo chmod +x /opt/scripts/hello.py
```

3. Add to crontab (run every 5 minutes):

```
*/5 * * * * /usr/bin/python3 /opt/scripts/hello.py
 >> /var/log/hello.log 2>&1
```

5. Recipe 04: Load a Configuration File

1. Create a non-secret configuration file:

```
sudo nano /etc/myapp/config.env
APP_ENV=production
APP_NAME=veriqta-app
LOG_DIR=/var/log/myapp
APP_PORT=8080
```

2. Use it in a script:

```
sudo nano /opt/scripts/app-task.sh
#!/bin/bash
source /etc/myapp/config.env
echo "Running $APP_NAME in $APP_ENV"
>> "$LOG_DIR/task.log"
```

3. Make it executable and add to crontab:

```
sudo chmod +x /opt/scripts/app-task.sh
0 * * * * /opt/scripts/app-task.sh
```

6. Example Crontab with Environment Variables

```
# Set required environment variables
PATH=/usr/local/bin:/usr/bin:/bin
APP_ENV=production
LOG_DIR=/var/log/myapp

# Run Python script every 5 minutes
*/5 * * * * /usr/bin/python3 /opt/scripts/hello.py \
  >> /var/log/hello.log 2>&1

# Run application task every hour
0 * * * * /opt/scripts/app-task.sh \
  >> /var/log/app-task.log 2>&1
```

7. Key Takeaways

- ✓ Cron does not load your shell profile files.
- ✓ Always use absolute paths (e.g. /usr/bin/python3).
- ✓ Check and set required environment variables.
- ✓ Use configuration files for non-secret settings.
- ✓ Test scripts manually before adding to cron.
- ✓ Always log output to verify execution.

Page 5: Keep Disks from Filling Up

VERIQTA

1. Why Disk Space Matters

- Applications, logs, temporary files and backups can quickly use disk space.
- When a disk is full, applications may fail, logs stop, and the server can become unstable.
- Use cron to monitor disk usage and clean up unnecessary files automatically.

2. Check Disk Usage with df

```
df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	50G	20G	28G	42%	/
/dev/sda2	100G	85G	10G	89%	/var
/dev/sdb1	200G	50G	150G	25%	/data

3. Recipe 05: Monitor Disk Usage and Alert

1. Create a disk check script:

```
sudo nano /usr/local/bin/check-disk.sh
#!/bin/bash
THRESHOLD=80
LOG_FILE=/var/log/disk-check.log

USAGE=$(df -h / | awk 'NR==2 {gsub("%","",$5); print $5}')
if [ "$USAGE" -ge "$THRESHOLD" ]; then
  echo "$(date '+%Y-%m-%d %H:%M:%S') - Disk usage
  is ${USAGE}% (threshold: ${THRESHOLD}%)" |
  tee -a "$LOG_FILE"

  # Example: send email alert (requires mail configured)
  echo "Disk usage is ${USAGE}'% on ${hostname)" |
  mail -s "Disk Alert" admin@example.com
fi
```

2. Make it executable:

```
sudo chmod +x /usr/local/bin/check-disk.sh
```

3. Add to cron (check every 10 minutes):

```
*/10 * * * * /usr/local/bin/check-disk.sh
```

4. Check the Log

```
tail -n 5 /var/log/disk-check.log
```

```
2026-10-24 10:00:01 - Disk usage is 85%
(threshold: 80%)
2026-10-24 10:10:01 - Disk usage is 88%
(threshold: 80%)
```

5. Recipe 06: Remove Expired Temporary Files

1. Dry run first (see what will be deleted):

```
sudo find /var/tmp/myapp -type f -name "*.log" \
        -mtime +7 -print
```

2. Delete files older than 7 days:

```
sudo find /var/tmp/myapp -type f -name "*.log" \
        -mtime +7 -delete
```

3. Run via cron (every day at 2 AM):

```
0 2 * * * /usr/bin/find /var/tmp/myapp -type f \
        -name "*.log" -mtime +7 -delete
```

6. Understand find Options

-type f	Match files (not directories)
-name "*.log"	Match files with .log extension
-mtime +7	Files modified more than 7 days ago
-print	Show matching files (use for dry run)
-delete	Delete matching files (use after testing)

7. Important Safety Rules

- Always run a dry run first with -print.
- Use the correct directory (do not use / or /var blindly).
- Set a clear retention limit (e.g. 7, 14 or 30 days).
- Avoid deleting files that are still in use.
- Be careful with wildcards like *.
- Test commands manually before adding to cron.
- Use absolute paths in cron jobs.

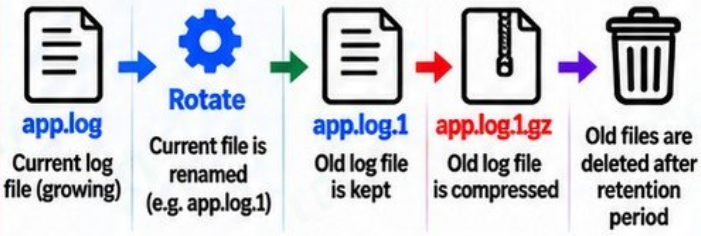
Page 6: Logs Need Maintenance, Not Blind Deletion

VERIQTA

1. Why Log Maintenance Matters

- Logs grow continuously and can fill up disks.
- Large log files affect application performance.
- Proper rotation, compression and retention keep your system healthy.
- Blindly deleting logs can cause data loss and may not free disk space if the file is still open by a running process.

2. How Log Rotation Works



3. Recipe 07: Rotate Application Logs Using logrotate

1. Create a logrotate configuration file:

```
sudo nano /etc/logrotate.d/myapp
/var/log/myapp/*.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 0640 myapp myapp
    sharedscripts
    postrotate
        systemctl reload myapp
    endscript
}
```

Configuration explained:

- **daily** - rotate logs every day
- **rotate 7** - keep 7 old logs
- **compress** - compress old logs
- **delaycompress** - compress from next rotation
- **missingok** - no error if log is missing
- **notifempty** - do not rotate empty logs
- **create 0640 myapp myapp** - create new log file with permissions
- **sharedscripts** - run postrotate once for all logs
- **postrotate** - reload application (after rotation)

2. Test the configuration (dry run):

```
sudo logrotate -d /etc/logrotate.d/myapp
```

3. Force rotation (for testing):

```
sudo logrotate -f /etc/logrotate.d/myapp
```

4. Verify Log Rotation

```
ls -lh /var/log/myapp/
-rw-r----- 1 myapp myapp  1.2M Oct 24 10:00 app.log
-rw-r----- 1 myapp myapp  800K Oct 23 10:00 app.log.1
-rw-r----- 1 myapp myapp  200K Oct 22 10:00 app.log.2.gz
```

5. Understanding Open File Handles

- A running application may keep writing to a log file even after it is deleted.
- The disk space will not be freed until the process closes the file.
- This is why log rotation is safer than manual deletion.



6. Recipe 08: Compress and Archive Completed Logs

1. Move and compress old logs (older than 30 days):

```
find /var/log/myapp -type f -name "*.log.*" \
  -mtime +30 -exec gzip {} \;
```

2. Move compressed logs to an archive directory:

```
sudo mkdir -p /var/log/myapp/archive
sudo find /var/log/myapp -type f -name "*.gz" \
 -mtime +30 -exec mv {} /var/log/myapp/archive/ \;
```

7. Retention and Cleanup Policies

- Decide how many days or files to keep (e.g. 7, 14, 30 days).
- Use logrotate options or a separate cleanup script.
- Keep important logs longer for auditing.
- Store archived logs on a dedicated disk if needed.

8. Key Takeaways

- Use logrotate instead of manual deletion.
- Compressed and archived logs save disk space.
- Deleting a log file may not free space if it is still open.
- Always test log rotation and cleanup with a dry run first.
- Set clear retention limits based on your requirements.
- Monitor disk usage regularly.

Page 7: Backups Are Only Useful When They Can Be Restored

VERIQTA

1. Why Backups Matter

- Configuration files, application data and system settings can be lost or corrupted.
- Backups help you recover from accidental deletions, failed deployments, server issues and security incidents.
- A successful backup command does not mean the backup is usable. You must test it.

2. Backup Workflow



3. Recipe 09: Create a Timestamped Backup

1. Create a backup directory:

```
sudo mkdir -p /backups/myapp
```

2. Create a compressed, timestamped archive:

```
sudo tar -czpf /backups/myapp/myapp-config-
$(date +%Y-%m-%d_%H-%M-%S).tar.gz /etc/myapp
```

3. List the created file:

```
ls -lh /backups/myapp/
```

4. What the tar Command Options Mean

Option	Description
-c	Create a new archive
-z	Compress using gzip
-p	Preserve file permissions
-f	Specify archive file name
-v	Show files being processed (optional)

5. Recipe 10: Verify Backup Integrity

1. Create a checksum for the backup file:

```
sha256sum /backups/myapp/myapp-config-2026-10-24_14-30-00.tar.gz \\
 > /backups/myapp/myapp-config-2026-10-24_14-30-00.tar.gz.sha256
```

2. Verify the checksum later:

```
sha256sum -c /backups/myapp/myapp-config-2026-10-24_14-30-00.tar.gz.sha256
```

3. Test extraction to confirm the backup is usable:

```
sudo mkdir -p /tmp/restore-test
sudo tar -xzf /backups/myapp/myapp-config-2026-10-24_14-30-00.tar.gz
  -C /tmp/restore-test
ls -l /tmp/restore-test
```

6. Store Backups in a Protected Location

- Use a dedicated backup directory (e.g. /backups).
- Ensure enough disk space and proper permissions.
- Consider a separate disk, mount or remote storage (e.g. NFS, S3).
- Restrict access to authorized users only.

Set permissions:

```
sudo chmod 700  /backups/myapp
sudo chown root:root  /backups/myapp
```

7. Example Output

```
$ ls -lh /backups/myapp/
total 12M
-rw-r--r-- 1 root root 3.4M Oct 24 14:30
myapp-config-2026-10-24_14-30-00.tar.gz
-rw-r--r-- 1 root root  102 Oct 24 14:30
myapp-config-2026-10-24_14-30-00.tar.gz.sha256
```

8. Important Points

- Use absolute paths in cron scripts.
- Include only necessary files and directories.
- Compress backups to save space.
- Always verify the backup with a checksum and test extraction.
- A successful command (exit code 0) does not guarantee a usable backup.
- Set a retention policy and delete old backups automatically.

9. Key Takeaways

- Use tar with the correct options for reliable backups.
- Create and verify checksums.
- Store backups in a protected location.
- Always test extraction to confirm the backup is usable.
- Implement a retention strategy.
- Automate and monitor your backup jobs with cron.
- Focus on recovery, not just file creation.

Page 8: Schedule Database Backups Correctly

VERIQTA

1. Why Database Backups Are Important

- Protects against data loss from accidental deletion, server failure, or corruption.
- Helps you recover from failed deployments, human errors and security incidents.
- A successful backup command does not mean the backup is usable. You must test the restore.

2. PostgreSQL Backup Workflow



3. Recipe 11: Back up PostgreSQL with pg_dump

1. Create a secure backup script:

```
sudo nano /usr/local/bin/pg-backup.sh
#!/bin/bash
DB_NAME="myapp"
DB_USER="backup_user"
BACKUP_DIR="/backups/postgresql"
TIMESTAMP=$(date +%Y-%m-%d_%H-%M-%S)

mkdir -p $BACKUP_DIR
PGPASSWORD=$(cat /etc/pg/backup.pass) \
pg_dump -U $DB_USER -h localhost -d $DB_NAME \
 -F c -f $BACKUP_DIR/${DB_NAME}-${TIMESTAMP}.dump
```

2. Make it executable:

```
sudo chmod +x /usr/local/bin/pg-backup.sh
```

3. Add to crontab (run daily at 2 AM):

```
0 2 * * * /usr/local/bin/pg-backup.sh >> /var/log/pg-backup.log 2>&1
```

4. Back Up with Restricted Credentials

- Use a dedicated database user with MINIMAL privileges (e.g. CONNECT, pg_dump access).
- Store credentials in a protected file (not in the script).
- Do not use the main application user or superuser.

/etc/pg/backup.pass

```
strong_password_here
```

```
sudo chmod 600 /etc/pg/backup.pass
```

5. Ensure a Consistent Backup

- pg_dump creates a consistent backup for most use cases.
- For large databases or high-write environments, consider running backups during a low-traffic window.
- If using multiple tables/databases, back up each one.
- Monitor the exit status and check logs for errors.

6. Recipe 12: Apply Retention Rules

1. List verified backup files (older than 7 days):

```
find /backups/postgresql -type f -name "*.dump" \
  -mtime +7 -print
```

2. Delete old backups (after verifying):

```
find /backups/postgresql -type f -name "*.dump" \
 -mtime +30 -delete
```

3. Keep compressed backups to save space:

```
find /backups/postgresql -name "*.dump" -mtime +1 -exec gzip {} \;
```

7. Verify Backup Integrity

1. Check file size (should be greater than a few KB/MB):

```
ls -lh /backups/postgresql/
```

2. Verify the backup file is valid:

```
pg_restore --list /backups/postgresql/myapp-2026-10-24_02-00-00.dump
```

3. Test restore (on a test server or temporary database):

```
createdb myapp_test
pg_restore -d myapp_test /backups/postgresql/myapp-2026-10-24_02-00-00.dump
```

8. Common Backup Failures

- Disk full (backup file is incomplte).
- Permission denied (wrong credentials).
- Network or database not reachable.
- Backup command succeeds but file is corrupt.

9. Key Takeaways

- Use pg_dump with a restricted database user.
- Store credentials securely and set proper permissions.
- Apply retention rules to remove old backups.
- Compress backups to save disk space.
- Always verify and test restore to confirm the backup is usable.
- Monitor logs and exit status to detect failures early.

Page 9: Know When Your Application Stops Responding

VERIQTA

1. Why Health Checks Are Important

- Detect application failures early.
- Helps you respond before users are affected.
- Validates that the application is actually working, not just listening on a port.
- Can be automated with cron to run at regular intervals.

2. How an HTTP Health Check Works



curl	Application	Result	Log / Alert
Send HTTP request to health endpoint	Returns HTTP status (e.g. 200 OK)	Check status and response time	Record result and notify if it fails

3. Recipe 13: Check an HTTP Health Endpoint

1. **Test the endpoint manually:**

```
curl -I https://example.com/health
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 15
```

2. **Check response with timeout (fail if not 200):**

```
curl -s -o /dev/null -w "%{http_code}" \
--max-time 10 https://example.com/health
```

3. **Example output:**

```
200   # application is healthy
500   # application returned an error
000   # connection failed or timed out
```

4. Important curl Options

Option	Description
-s	Silent mode (no progress output)
-o /dev/null	Ignore the response body
-w "%{http_code}"	Print HTTP status code only
--max-time 10	Set a timeout (in seconds)
-I	Send a HEAD request (headers only)
-L	Follow redirects

5. Understanding Exit Codes and Status

Result	Meaning
Exit code 0 + HTTP 200	Application is healthy
Exit code 0 + HTTP 4xx/5xx	Application returned an error
Exit code 7	Failed to connect to the server
Exit code 28	Request timed out
HTTP 301/302	Redirected (may be normal)
HTTP 401/403	Access denied (check credentials)

6. Recipe 14: Record Failures and Notify Operator

1. **Create a health check script:**

```
sudo nano /usr/local/bin/check-app.sh
#!/bin/bash
URL="https://example.com/health"
LOG_FILE="/var/log/app-health.log"
TIMEOUT=10

HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" \
          --max-time $TIMEOUT "$URL")

if [ "$HTTP_CODE" != "200" ]; then
   echo "$(date '+%Y-%m-%d %H:%M:%S') - HTTP $HTTP_CODE" \
   >> $LOG_FILE
   echo "Application health check failed (HTTP $HTTP_CODE)" |
   mail -s "App Alert" admin@example.com
fi
```

2. **Make it executable:**

```
sudo chmod +x /usr/local/bin/check-app.sh
```

3. **Add to crontab (check every 5 minutes):**

```
*/5 * * * * /usr/local/bin/check-app.sh
```

7. Why a Successful TCP Connection Is Not Enough

- A server can accept TCP connections but the application may be crashed or misconfigured.
- The port may be open but the application may return an error (HTTP 500).
- A load balancer or reverse proxy may respond, but the backend application is down.
- You must check the actual application endpoint and validate the HTTP status code.

8. Key Takeaways

- Use curl with a timeout to avoid hanging scripts.
- Always check the HTTP status code, not just the connection.
- Log failures with a timestamp.
- Notify an operator (email, Slack, etc.) when checks fail.
- Test your health check script manually.
- Monitor and alert regularly using cron.

Page 10: Run Scheduled Application Work

VERIQTA

1. Why Schedule Application Work

- Run reports, data processing and other tasks without manual intervention.
- Keeps operations consistent and reliable.
- Reduces human error and saves time.
- Allows you to run tasks during low-traffic hours.
- Helps with cleanup of temporary and old files generated by the application.

2. Typical Scheduled Application Workflow



3. Recipe 15: Execute a Scheduled Appllication Report

1. Create the report script:

```
sudo nano /opt/myapp/scripts/generate-report.sh
#!/bin/bash
set -euo pipefail
APP_DIR="/opt/myapp"
LOG_FILE="/var/log/myapp/report.log"
DATE=$(date +%Y-%m-%d_%H-%M-%S)

cd "$APP_DIR"
/usr/bin/python3 /opt/myapp/reports/generate_report.py \
 --output /opt/myapp/reports/report_${DATE}.csv \
 >> "$LOG_FILE" 2>&1
```

2. Make it executable:

```
sudo chmod +x /opt/myapp/scripts/generate-report.sh
```

3. Add to crontab (run daily at 1 AM as application user):

```
sudo -u myapp crontab -e

0 1 * * * /opt/myapp/scripts/generate-report.sh
```

4. Important Points for Scheduled Tasks

- Use the correct working directory (cd to your app directory).
- Run as the appropriate application user (avoid root).
- Use absolute paths for all commands, scripts and files.
- Validate input parameters before processing.
- Log output and errors to a file.
- Check return codes and alert if the task fails.
- Run heavy tasks during off-peak hours.

5. Recipe 16: Clean Up Expired Application Files

1. Dry run first (see what will be deleted):

```
sudo find /opt/myapp/data/exports -type f \
  -name "*.csv" -mtime +30 -print
```

2. Delete files older than 30 days (safe deletion):

```
sudo find /opt/myapp/data/exports -type f \
  -name "*.csv" -mtime +30 -delete
```

3. Add to crontab (run daily at 2 AM):

```
0 2 * * * /usr/bin/find /opt/myapp/data/exports \
  -type f -name "*.csv" -mtime +30 -delete
```

6. Avoid Deleting Files Still in Use

- Do not delete files that are currently being written by the application.
- Use file age (e.g. -mtime +30) to avoid recent files.
- Keep enough retention days based on your requirements.
- Check open file handles if needed:

```
sudo lsof | grep /opt/myapp/data/exports
```

- Consider rotating or moving files instead of direct deletion in some applications.

7. Example: Report Output and Cleanup

Report output (log file):

```
2026-10-24 01:00:01  INFO  Report
started
2026-10-24 01:00:15  INFO  Report
completed: report_2026-10-24_01-00-00.csv
```

Cleanup output (dry run):

```
/opt/myapp/data/exports/
report_2026-09-10.csv
/opt/myapp/data/exports/
report_2026-09-12.csv
/opt/myapp/data/exports/
report_2026-09-14.csv
```

8. Key Takeaways

- Use absolute paths and the correct working directory.
- Run scheduled tasks as a dedicated application user.
- Validate inputs and check return codes.
- Log all output and errors.
- Be careful with file deletion and use retention limits.
- Test your tasks manually before adding them to cron.

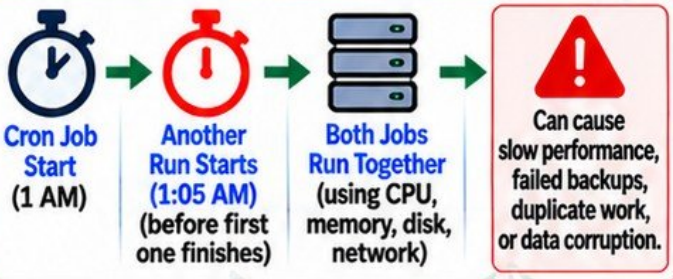
Page 11: Prevent Two Cron Jobs from Running Together

VERIQTA

1. Why Prevent Overlapping Executions

- Overlapping cron jobs can consume high CPU, memory, disk and network resources.
- Can cause data corruption, duplicate work or failed backups.
- Prevents race conditions where two jobs run at the same time.
- Important for long-running tasks like backups, reports and data processing.

2. What Happens Without a Lock



3. Recipe 17: Use flock to Prevent Overlapping Executions

1. Create a script with flock:

```
sudo nano /usr/local/bin/backup.sh
#!/bin/bash
# Your backup commands here
echo "Starting backup at $(date)"
# ... backup logic
echo "Backup completed at $(date)"
```

2. Add execute permission:

```
sudo chmod +x /usr/local/bin/backup.sh
```

3. Use flock in cron (non-blocking, skip if already running):

```
0 2 * * * /usr/bin/flock -n /var/lock/backup.lock
/usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
```

4. Alternative (wait until lock is free):

```
0 2 * * * /usr/bin/flock /var/lock/backup.lock \
/usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
```

4. Understanding flock Options

Option	Description
-n	Do not wait. Exit if the lock is already held (useful for cron jobs).
(no option)	Wait until the lock is free.
-w 30	Wait up to 30 seconds for the lock.
-x	Exclusive lock (default for flock).
-s	Shared lock (rarely used for cron).

5. Recipe 18: Apply a Runtime Limit with timeout

1. Run the script with a timeout (e.g. 30 minutes):

```
0 2 * * * /usr/bin/flock -n /var/lock/backup.lock \
/usr/bin/timeout 30m /usr/local/bin/backup.sh \
>> /var/log/backup.log 2>&1
```

2. Capture the exit status:

```
/usr/bin/timeout 30m /usr/local/bin/backup.sh
echo "Exit code: $?"
```

3. Check timeout exit codes:
   - Exit code 0 - script completed successfully.
   - Exit code 124 - command was terminated by timeout.
   - Other codes - script failed (check logs).

6. What Are Race Conditions and Hung Processes

- Race condition: two instances of the same job run at the same time and interfere with each other (e.g. writing to the same files or database).
- Hung process: a job gets stuck and never finishes, keeping system resources in use.
- Overlapping backups can exhaust CPU, memory, disk I/O and network bandwidth, causing service degradation or failure.
- Using flock and timeout helps prevent these issues.

7. Monitor and Verify

1. Check running jobs: `ps aux | grep backup.sh | grep -v grep`
2. Check lock file (if it exists): `ls -l /var/lock/backup.lock`
3. Check logs for skipped runs or timeouts: `tail -f /var/log/backup.log`

8. Example Log Output

```
2026-10-24 02:00:01   Starting backup...
2026-10-24 02:10:15   Backup completed successfully
Exit code: 0

2026-10-25 02:00:00   Another run skipped
(lock is held)

2026-10-26 02:00:00   Backup timed out after 30m
Exit code: 124
```

9. Key Takeaways

- Use flock to prevent overlapping executions.
- Use timeout to stop hung processes.
- Always capture and check the exit status.
- Overlapping backups can exhaust server resources.
- Monitor logs regularly.
- Adjust timeout values based on your environment.

Page 12: Build a Safer Retry Strategy

VERIQTA

1. Why Use a Retry Strategy?

- Network and external services can fail temporarily even when everything is correct.
- Retries help recover from transient failures without manual intervention.
- A good retry strategy uses a limited number of attempts with backoff delays.
- Retries must be designed carefully to avoid duplicating actions (e.g. payments, messages, database changes).

2. Transient vs Permanent Failures

Transient Failures (Retry Allowed)	Permanent Failures (Do Not Retry)
✓ Network timeouts	✗ Invalid input (HTTP 400)
✓ Temporary service unavailability (HTTP 503)	✗ Authentication failure (HTTP 401/403)
✓ Rate limits (HTTP 429)	✗ Resource not found (HTTP 404)
✓ DNS resolution issues	✗ Permission denied
✓ Short-lived database connection errors	✗ Application error due to bad data or logic

3. Recipe 19: Retry a Network Operation

1. Create a script with retry and backoff:

```
sudo aano /usr/local/bin/retry-curl.sh
#!/bin/bash
URL="https://example.com/api/health"
MAX_ATTEMPTS=3
SLEEP_SECONDS=10
attemp=1
while [ $attempt -le $MAX_ATTEMPTS ]; do
  echo "Attempt $attempt: Checking $URL"
  if curl -s -f --max-time 10 "$URL" ; then
    echo "Success on attempt $attempt"
    exit 0
  else
    echo "Failed on attempt $attempt"
    if [ $attempt -lt $MAX_ATTEMPTS ]; then
      echo "Retrying in ${SLEEP_SECONDS}s..."
      sleep $SLEEP_SECONDS
    fi
  fi
  attempt=$((attempt + 1))
done
echo "All $MAX_ATTEMPTS attempts failed"
exit 1
```

2. Make it executable:

```
sudo chmod +x /usr/local/bin/retry-curl.sh
```

3. Add to crontab (run every 10 minutes):

```
*/10 * * * * /usr/local/bin/retry-curl.sh \
 >> /var/log/retry-curl.log 2>&1
```

4. Retry Backoff Strategies

Strategy	Example Delay Sequence	Use Case
Fixed delay	10s, 10s, 10s	Simple, predictable
Linear backoff	10s, 20s, 30s	Moderate load
Exponential backoff	5s, 10s, 20s, 40s	Reduces pressure on failing services
Exponential backoff with jitter	5s, 12s, 25s, 50s (with randonmess)	Best for real systems

5. Recipe 20: Record Failed Runs

1. Append failures to a log file with timestamp:

```
echo "$(date '+%Y-%m-%d %H::%M:%S') - Operation failed" \
  >> /var/log/myapp-retry.log
```

2. Send a notification on repeated failure (example using email):

```
if [ $attempt -eq $MAX_ATTEMPTS ]; then
  echo "Operation failed after $MAX_ATTEMPTS attempts" | \
  mail -s "Cron Alert: Operation Failed" admin@exemple.com
fi
```

6. Important Considerations

- ✓ Ensure the operation is idempotent (safe to run multiple times).
- ✓ Validate input data before retrying.
- ✓ Use a reasonable number of attempts and backoff delays.
- ✓ Log failures for later investigation.
- ✓ Do not retry operations that can cause duplicate payments, messages, or database changes unless the operation is safe.
- ✓ Consider alerting an operator if retries keep failing.

7. Why Retries Can Be Dangerous

- Retries can duplicate payments or send the same message multiple times.
- They can create multiple database records if the operation is not idempotent.
- They can increase load on a failing service and make the problem worse.
- Always understand the operation before adding retries.

8. Key Takeaways

- ✓ Use limited retry attempts with backoff.
- ✓ Understand the difference between transient and permanent failures.
- ✓ Ensure operations are idempotent.
- ✓ Record and alert on failed runs.
- ✓ Avoid endless retries that can cause duplicate actions.

Page 13: Schedule Docker Maintenance Without Breaking Containers

VERIQTA

1. Why Monitor and Clean Docker?

- Helps you track container health, resource usage and storage consumption.
- Prevents disk space issues caused by unused images, containers and volumes.
- Allows you to plan safe cleanup without breaking running containers.
- Keeps your Docker host stable and reliable in production.

2. Docker Components and Storage



Containers	Images	Volumes	Networks	Storage
Running or stopped applications	Read-only templates for containers	Persistent data for containers	Enable container communication	All of these consume disk space

3. Recipe 21: Record Docker Container Health and Status

1. List all containers with health status:

```
docker ps -a --format \
'table {{.Names}}\t{{.Status}}\t{{.Ports}}\t{{.Image}}'
```

2. Inspect a specific container (detailed info):

```
docker inspect <container_name> --format \
'{{json .State.Health}}'
```

3. Check container resource usage (live):

```
docker stats --no-stream
```

4. Save container report to a file:

```
docker ps -a --format \
'table {{.Names}}\t{{.Status}}\t{{.Image}}' \
> /var/log/docker-containers-$(date +%Y-%m-%d).log
```

4. Understanding Container Health Status

Status	Meaning	Action
healthy	Container is running and healthy	No action needed
unhealthy	Health check is failing	Investigate logs and fix the issue
running	Container is running (but may not have a health check)	Check application manually if needed
exited	Container has stopped	Investigate why it exited
restarting	Container is repeatedly restarting	Check logs and fix the root cause

5. Recipe 22: Report Unused Docker Resources

1. Show unused resources (safe - no deletion):

```
docker system df
docker image ls -f "dangling=true"
docker container ls -a -f "status=exited"
```

2. Detailed view of space usage:

```
docker system df -v
```

3. Save report to a file:

```
docker system df -v > /var/log/docker-usage-$(date +%Y-%m-%d).log
```

6. Important Considerations

- Docker commands usually require root or a user in the docker group.
- Always use absolute paths in cron jobs.
- Monitor disk usage regularly to prevent the server from running out of space.
- Do not automatically delete images, containers or volumes without verification.
- Unused resources may still be needed (e.g. for rollback).
- Log output and errors to a file for later investigation.

7. Example Output: docker system df

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	5	3	2.1GB	800MB (37%)
Containers	4	2	1.2GB	600MB (50%)
Local Volumes	6	3	5.6GB	2.4GB (42%)
Build Cache	12	0	1.8GB	1.8GB (100%)

8. Why Indiscriminate Pruning Is Dangerous

- Can remove images needed for running containers.
- May delete stopped containers that are required for rollback.
- Can remove volumes containing important data.
- Might break applications if the wrong resources are deleted.
- Always review what will be deleted before using prune commands.

9. Key Takeaways

- Monitor container health and status regularly.
- Track Docker storage usage.
- Report unused resources without automatic deletion.
- Use the correct permissions and absolute paths in cron.
- Avoid indiscriminate pruning to prevent accidental data loss.

Page 14: Use Cron for Cloud Operations Carefully

VERIQTA

1. Why Use Cron for Cloud Operations?

- Automates regular reporting and inventory collection.
- Helps you monitor resources and costs.
- Provides visibility without manually logging into the AWS console.
- Uses read-only access to keep your cloud environment safe.
- Supports proactive management and early detection of issues.

2. Key AWS Concepts



IAM
Controls who can access AWS resources



Least Privilege
Give only the permissions required (read-only)



Credentials
Use IAM role or access keys stored securely (e.g. ~/.aws/ credentials)



Reporting vs Changes
Reporting collects data Changing infrastructure modifies resources

3. Recipe 23: Scheduled AWS Resource Inventory

1. Create a read-only IAM policy (example):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["ec2:Describe*", "s3:List*",
       "rds:Describe*", "lambda:List*"],
      "Resource": "*"
    }
  ]
}
```

2. Run a resource inventory script:

```
aws ec2 describe-instances --output table > \
/var/log/aws-inventory-$(date +%Y-%m-%d).txt
```

3. Make it executable:

```
sudo chmod +x /usr/local/bin/aws-inventory.sh
```

4. Add to crontab (run daily at 2 AM):

```
0 2 * * * /usr/local/bin/aws-inventory.sh \
 >> /var/log/aws-inventory.log 2>&1
```

4. Recipe 24: Daily AWS Cost or Usage Report

1. Use AWS Cost Explorer (example):

```
aws ce get-cost-and-usage \
  --time-period Start=$(date -d "yesterday" +%Y-%m-%d),
   End=$(date -d "yesterday" +%Y-%m-%d) \
  --granularity DAILY \
  --metrics "UnblendedCost"
  --output table > /var/log/aws-cost-$(date +%Y-%m-%d).txt
```

2. Alternative: Generate a simple CSV report:

```
aws ce get-cost-and-usage \
  --time-period Start=$(date -d "yesterday"
  +%Y-%m-%d),End=$(date -d "yesterday" +%Y-%m-%d)
  --granularity DAILY --metrics "UnblendedCost"
  --output csv > /var/log/aws-cost-$(date +%Y-%m-%d).csv
```

3. Add to crontab (run daily at 3 AM):

```
0 3 * * * /usr/local/bin/aws-cost-report.sh \
 >> /var/log/aws-cost.log 2>&1
```

5. Handling API Failures and Errors

```
#!/bin/bash
OUTPUT="/var/log/aws-inventory.log"
if ! aws ec2 describe-instances >> $OUTPUT 2>&1; then
  echo "$(date): AWS API request failed" >> $OUTPUT
  exit 1
fi
```

6. Important Considerations

- Use a read-only IAM role or user with least privilege.
- Store credentials securely (use IAM role, not hardcoded keys).
- Handle API failures with logging and alerts.
- Be aware of API rate limits.
- Reporting collects information only, it does not change infrastructure.
- Avoid using powerful permissions that can modify or delete resources.

7. Example Output: AWS Resource Inventory

InstanceId	InstanceType	State	PublicIpAddress	Name
i-0a1b2c3d4e5f	t3.micro	running	3.24.12.10	web-server
i-0f6g7h8i9j0k	t3.small	stopped	-	test-server
i-1a2b3c4d5e6f	t3.medium	running	18.206.11.44	app-server

8. Example Output: AWS Cost Report (CSV)

```
Date,Service,UsageType,Usage,UnblendedCost
2026-10-24,AmazonEC2,BoxUsage,100,12.45
2026-10-24,AmazonS3,TimedStorage-ByteHrs,500000,1.23
2026-10-24,AmazonRDS,InstanceUsage,24,8.76
```

9. Key Takeaways

- Use cron for regular inventory and cost reporting.
- Apply least privilege IAM permissions.
- Keep credentials secure (prefer IAM roles).
- Log and monitor API failures.
- Understand the difference between reporting and changing infrastructure.
- Test your scripts manually before adding them to cron.

Page 15: Cron Versus Kubernetes CronJobs

VERIQTA

1. Why Use Kubernetes CronJobs?

- Run scheduled tasks inside the Kubernetes cluster.
- Useful for batch jobs, data processing, backups and maintenance.
- Uses containers, so the same environment runs every time.
- Supports retry, history and concurrency control.
- Scales and is managed by the cluster (no need to manage cron on each node).

2. Kubernetes CronJob Workflow



Schedule (e.g. every 5 minutes) → **CronJob** creates a Job → **Job** runs a Pod with your task → **Job History** keeps success and failed runs

3. Recipe 25: Create a Kubernetes CronJob

1. Create a CronJob YAML file (backup.yaml):

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: app-backup
spec:
  schedule: "0 2 * * *"  # run daily at 2 AM
  concurrencyPolicy: Forbid
  startingDeadlineSeconds: 300
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: alpine:latest
            command: ["/bin/sh", "-c", "echo Backup
              started; tar -czf /data/backup.tar.gz
              /data; echo Backup completed"]
          restartPolicy: OnFailure
```

2. Apply the CronJob:

```
kubectl apply -f backup.yaml
```

4. Verify and Inspect

1. Check the CronJob: `kubectl get cronjob`
2. Check created jobs: `kubectl get jobs --sort-by=.metadata.creationTimestamp`
3. Check job pods: `kubectl get pods --selector=job-name=app-backup-<job-id>`
4. View logs: `kubectl logs <pod-name>`

5. Important CronJob Fields

Field	Description
schedule	Cron expression (e.g. "0 2 * * *" for daily at 2 AM).
concurrencyPolicy	Allow, Forbid or Replace (controls overlapping runs).
startingDeadlineSeconds	How long to wait before marking a run as failed.
successfulJobsHistoryLimit	Number of successful jobs to keep.
failedJobsHistoryLimit	Number of failed jobs to keep.

6. Cron vs Kubernetes CronJobs

Feature	Linux Cron	Kubernetes CronJob
Runs on	Single server/host	Kubernetes cluster
Environment	Uses host environment	Runs in a container
Management	Manual on each server	Managed by Kubernetes
Scalability	Limited to one server	Runs across the cluster
Reliability	Can be affected by server issues	More reliable with cluster control plane
Logging	System logs (e.g. /var/log/cron)	kubectl logs and Kubernetes events
Best for	Simple host tasks	Containerized, scalable and portable tasks

7. Concurrency Policy Options

Policy	Behavior	Use Case
Allow	Multiple jobs can run at the same time.	Use when tasks are independent and can run in parallel.
Forbid	Skip a new run if the previous job is still running.	Use for backups or tasks that should not overlap.
Replace	Terminate the running job and start a new one.	Use when only the latest run matters.

8. Troubleshooting Common Issues

- **CronJob not running:** check the schedule, timezone and YAML syntax.
- **Job fails immediately:** check container image, command and permissions.
- **Pod in CrashLoopBackOff:** check logs with kubectl logs.
- **Missed runs:** check startingDeadlineSeconds.
- **Too many old jobs:** adjust successfulJobsHistoryLimit and failedJobsHistoryLimit.
- **Image pull errors:** verify the image name and access (if private registry).
- **Check events:** kubectl describe cronjob <name>.

9. Key Takeaways

- Use Kubernetes CronJobs for scheduled tasks in a containerized environment.
- Understand schedule, concurrencyPolicy, startingDeadlineSeconds and job history settings.
- Monitor jobs and logs regularly and set appropriate history limits.
- Use Linux cron for simple host tasks and Kubernetes CronJobs for scalable cluster workloads.
- Always test your CronJob manually before relying on it in production.

Page 16: Cron Must Leave Evidence

VERIQTA

1. Why Logging Is Important?

- Shows what the cron job did, when it ran and whether it succeeded or failed.
- Helps with troubleshooting and root cause analysis.
- Provides evidence for auditing and monitoring.
- Prevents silent failures, which can go unnoticed for a long time.
- Allows you to detect and fix issues quickly before they cause bigger problems.

2. What to Capture in Logs

Standard Output (stdout)
Normal output from the command (e.g. success messages).

Standard Error (stderr)
Error messages and warnings.

Timestamps
When the job started and finished.

Exit Status
Whether the job succeeded (0) or failed (non-zero code).

3. Recipe 26: Capture stdout, stderr, Timestamps and Exit Status

1. Create a script with proper logging:

```
#!/bin/bash
LOG_FILE="/var/log/myapp-cron.log"

echo "[$(date '+%Y-%m-%d %H:%M:%S')] Starting job"
# Run your command
/usr/local/bin/myapp-task.sh >> "$LOG_FILE" 2>&1

EXIT_CODE=$?
echo "[$(date '+%Y-%m-%d %H:%M:%S')] Job finished \
with exit code: $EXIT_CODE" >> "$LOG_FILE"
```

2. Make it executable:

```
sudo chmod +x /usr/local/bin/myapp-cron.sh
```

3. Add to crontab (run daily at 1 AM):

```
0 1 * * * /usr/local/bin/myapp-cron.sh
```

4. Understanding Exit Status Codes

Exit Code	Meaning
0	Command executed successfully.
1	General error (check logs for details).
2	Misuse of shell command or invalid input.
126	Command found but not executable.
127	Command not found.
130	Command terminated by Ctrl+C.

5. Recipe 27: Structured Execution Log

1. Create a script that logs in structured format (JSON-like):

```
#!/bin/bash
LOG_FILE="/var/log/myapp-cron-structured.log"

START_TIME=$(date '+%Y-%m-%dT%H:%M:%S')

echo "{\"time\":\"$START_TIME\", \"event\":\"start\"}" >>
    "$LOG_FILE"

/usr/local/bin/myapp-task.sh >> "$LOG_FILE" 2>&1
EXIT_CODE=$?
END_TIME=$(date '+%Y-%m-%dT%H:%M:%S')

echo "{\"time\":\"$END_TIME\", \"event\":\"end\", \
\"exit_code\":$EXIT_CODE}" >> "$LOG_FILE"
```

2. Add to crontab:

```
0 2 * * * /usr/local/bin/myapp-cron-structured.sh
```

6. Log File Example

```
2026-10-24  01:00:01  Starting job
2026-10-24  01:00:05  Processing data...
2026-10-24  01:00:10  Data processed successfully
2026-10-24  01:00:10  Job finished with exit code: 0

2026-10-25  01:00:01  Starting job
2026-10-25  01:00:03  Error: failed to connect to API
2026-10-25  01:00:03  Job finished with exit code: 1
```

7. Redirection Options

Command	Description
> file	Redirect stdout (overwrite file)
>> file	Redirect stdout (append to file)
2> file	Redirect stderr (overwrite file)
2>> file	Redirect stderr (append to file)
>> file 2>&1	Redirect both stdout and stderr (append)

8. Common Failure Diagnosis Steps

- Check the log file for error messages.
- Verify the exit status code.
- Confirm that the command and paths are correct.
- Check environment differences (cron vs manual run).
- Ensure the required permissions and credentials are available.
- Look for network, disk or resource issues.

9. Key Takeaways

- Always capture stdout, stderr, timestamps and exit status.
- Use structured logs for easier monitoring and analysis.
- Check logs regularly and set up alerts for failures.
- Do not assume the job succeeded — always verify the outcome.
- Good logging makes troubleshooting faster and prevents bigger issues.

Page 17: Alert When Automation Fails or Never Starts

VERIQTA

1. Why Alert on Failures and Missed Runs?

- Helps you detect problems quickly before they become bigger incidents.
- Cron jobs can fail due to errors, network issues, permission problems or missing dependencies.
- A job may never start (e.g. cron misconfiguration, server down), so you must also detect missed runs.
- Alerts ensure someone is notified instead of assuming everything is working.
- Silence is not proof of success.

2. Common Alerting Methods



Email (SMTP)
Simple and widely used.



Slack
Send alerts to a channel using a webhook.



Telegram
Send alerts using a bot and API token.



Monitoring Tools
Integrate with Prometheus, Grafana, etc.

3. Recipe 28: Alert on Unsuccessful Execution

1. Create a script with error alert (email example):

```
#!/bin/bash
LOG_FILE="/var/log/myapp-task.log"
EMAIL="admin@example.com"

/usr/local/bin/myapp-task.sh >> "$LOG_FILE" 2>&1
EXIT_CODE=$?

if [ $EXIT_CODE -ne 0 ]; then
   SUBJECT="ALERT: myapp-task failed on $(hostname)"
   BODY="Cron job myapp-task failed with exit code:
   $EXIT_CODE\n\nCheck logs: $LOG_FILE"
   echo -e "$BODY" | mail -s "$SUBJECT" "$EMAIL"
fi
```

2. Make it executable:

```
sudo chmod +x /usr/local/bin/myapp-task.sh
```

3. Add to crontab (run daily at 1 AM):

```
0 1 * * * /usr/local/bin/myapp-task.sh
```

4. Recipe 29: Detect Missing Scheduled Execution

1. Update a last-success timestamp after a successful run:

```
#!/bin/bash
LOG_FILE="/var/log/myapp-task.log"
SUCCESS_FILE="/var/lib/myapp-task.lastsuccess"

/usr/local/bin/myapp-task.sh >> "$LOG_FILE" 2>&1
if [ $? -eq 0 ]; then
   date '+%Y-%m-%d %H:%M:%S' > "$SUCCESS_FILE"
fi
```

2. Create a separate script to check for missed runs:

```
#!/bin/bash
SUCCESS_FILE="/var/lib/myapp-task.lastsuccess"
MAX_AGE_HOURS=26  # expected to run every 24 hours

if [ ! -f "$SUCCESS_FILE" ]; then
   echo "ALERT: myapp-task has never succeeded." | \
      mail -s "ALERT: Missing cron job" admin@example.com
   exit 1
fi

LAST_SUCCESS=$(stat -c %Y "$SUCCESS_FILE")
NOW=$(date +%s)
AGE_HOURS=$(( (NOW - LAST_SUCCESS) / 3600 ))

if [ $AGE_HOURS -gt $MAX_AGE_HOURS ]; then
   echo "ALERT: myapp-task has not run in $AGE_HOURS hours." | \
      mail -s "ALERT: Cron job missed" admin@example.com
fi
```

3. Add the check script to crontab (run every hour):

```
0 * * * * /usr/local/bin/check-myapp-task.sh
```

5. Example Alert Email

Subject: ALERT: myapp-task failed on server1

Cron job myapp-task failed with exit code: 1
Check logs: /var/log/myapp-task.log
Time: 2026-10-24 01:00:01
Host: server1

6. Common Reasons for Failures or Missed Runs

- Script errors or invalid commands.
- Network or API failures.
- Permission issues.
- Missing dependencies or environment variables.
- Cron not running or misconfigured.
- Server downtime or reboot.
- Job takes too long and overlaps with next run.

7. Key Takeaways

- Always alert when a job exits with a non-zero status.
- Track the last successful run and detect if a job never starts or misses a schedule.
- Set clear thresholds for missed runs based on the expected schedule.
- Send alerts to a real person or channel (email, Slack, Telegram, etc.).
- Check logs and investigate the root cause.
- Do not assume success because you did not receive an error.

Page 18: Secure Your Cron Jobs Before Production

VERIQTA

1. Why Security Matters for Cron Jobs?

- Cron jobs often run with high privileges (sometimes root).
- Can expose sensitive data (API keys, database passwords).
- Scripts may be modified if permissions are weak.
- Untrusted paths can execute malicious commands.
- Insecure cron jobs are a common target for attackers.
- Use least privilege and follow security best practices.

2. Common Security Risks



Exposed Secrets
API keys, passwords or tokens in scripts or environment files.



Unsafe Permissions
World-writable scripts or cron directories.



Untrusted Paths
Using relative paths or untrusted directories.



Unnecessary Root Access
Running jobs as root when not required.

3. Recipe 30: Audit Existing Cron Jobs

1. List current user crontab:

```
crontab -l
```

2. List system-wide cron files:

```
ls -l /etc/cron.d/
ls -l /etc/crontab
```

3. Check permissions of cron files and scripts:

```
ls -l /etc/cron.d/
ls -l /path/to/your/script.sh
```

4. Look for unsafe permissions (world-writable):

```
find /etc/cron.d /etc/cron.daily /etc/cron.hourly \
 /etc/cron.weekly /etc/cron.monthly -type f -perm -002
```

5. Check for secrets in scripts (example):

```
grep -Ei "(password|secret|api[_-]?key|token)" \
 /path/to/your/script.sh
```

4. Understanding Cron File Locations

Location	Used For	Notes
User crontab	Per-user jobs	Edit with crontab -e Runs with that user's privileges
/etc/crontab	System-wide jobs	Includes user field Uses system environment
/etc/cron.d/	Additional system jobs	Files with specific format Include user field
/etc/cron.daily/	Daily scripts	Run by run-parts (as root)
/etc/cron.hourly/	Hourly scripts	Run by run-parts (as root)
/etc/cron.weekly/	Weekly scripts	Run by run-parts (as root)
/etc/cron.monthly/	Monthly scripts	Run by run-parts (as root)

5. Replace an Unsafe Root Job with a Service Account

1. Create a dedicated service account:

```
sudo useradd -r -s /bin/bash appbackup
```

2. Give minimal permissions to script and directories:

```
sudo mkdir -p /opt/appbackup
sudo chown appbackup:appbackup /opt/appbackup
sudo chmod 750 /opt/appbackup
```

3. Move the script and set ownership:

```
sudo mv /root/backup.sh /opt/appbackup/backup.sh
sudo chown appbackup:appbackup /opt/appbackup/backup.sh
sudo chmod 700 /opt/appbackup/backup.sh
```

4. Update crontab to run as the service account:

```
0 2 * * * appbackup /opt/appbackup/backup.sh \
>> /var/log/appbackup.log 2>&1
```

6. Script Integrity and Safe Practices

- Use absolute paths (e.g. /usr/local/bin/script.sh).
- Set secure permissions (e.g. chmod 700 for scripts).
- Ensure script ownership is correct.
- Validate and sanitize inputs (avoid untrusted variables).
- Do not store secrets in plain text (use environment variables or secret management tools).
- Run with the lowest privileges required.
- Log output and monitor for failures.

7. Example: Before and After

✗ Unsafe (Running as Root)

```
0 2 * * * root /root/backup.sh
>> /var/log/backup.log 2>&1
```

Problems:
- Runs as root
- Script in /root (high risk)
- May contain exposed secrets
- Broad filesystem access

✓ Safer (Using Service Account)

```
0 2 * * * appbackup \
 /opt/appbackup/backup.sh
>> /var/log/appbackup.log 2>&1
```

Improvements:
- Runs with dedicated user
- Script in secure directory
- Least privilege access
- Better isolation and security

8. Key Takeaways

- ✓ Audit existing cron jobs for unsafe permissions and secrets.
- ✓ Use user crontabs or dedicated service accounts.
- ✓ Keep scripts in secure directories with correct ownership.
- ✓ Avoid running jobs as root unless absolutely necessary.
- ✓ Use absolute paths and validate inputs.
- ✓ Implement logging and monitoring.
- ✓ Regularly review and update cron jobs.

Page 19: The Cron Troubleshooting Playbook

VERIQTA

1. Evidence-First Troubleshooting Approach

1. **Confirm the problem:** what should happen vs what actually happened.
2. **Check the cron configuration:** user crontab and /etc/cron.d.
3. **Verify the cron service:** is it running and healthy.
4. **Inspect logs:** system logs and your script logs.
5. **Reproduce the environment:** run the command as the cron user.
6. **Check exit codes and output:** look for errors, warnings or empty results.

2. Useful Diagnostic Commands

`crontab -l`	View current user cron jobs
`sudo crontab -l`	View root cron jobs
`ls -l /etc/cron.d/`	List system cron files
`systemctl status cron`	Check cron service status
`journalctl -u cron`	View cron service logs
`grep CRON /var/log/syslog`	View cron related system logs
`tail -f /var/log/cron`	Follow cron logs (some systems)
`id`	Check current user
`which <command>`	Verify command path
`echo $PATH`	Check environment PATH

3. Incident 1: Job Runs Manually but Not from Cron

Possible Causes:

- Wrong user (different environment).
- Command not in PATH.
- Missing environment variables.
- Script not executable.
- Output redirected and errors hidden.

heck and Fix:

```
# Check cron job
crontab -l
# Check environment when run by cron
env > /tmp/cron-env.txt

# Use absolute paths
/usr/local/bin/script.sh

# Redirect output to a log file
0 2 * * * /usr/local/bin/script.sh >>
  /var/log/myjob.log 2>&1
f
```

4. Incident 2: Job Executes at the Wrong Time

Possible Causes:

- Wrong cron time zone.
- Server time different from expected.
- Daylight saving time changes.
- Multiple cron entries.

Check and Fix:

```
# Check server time
date
timedatectl
# Check cron time zone (if set)
cat /etc/timezone
# Check cron entry
crontab -l
# Example: run at 2 AM daily
0 2 * * * /usr/local/bin/script.sh
# Specify timezone if needed
CRON_TZ=UTC
0 2 * * * /usr/local/bin/script.sh
```

5. Incident 3: Script Fails with Permission Denied

Possible Causes:

- Script is not executable.
- Wrong file or directory permissions.
- Running as a different user.
- Accessing a directory without write permission.
- Using relative paths.

Check and Fix:

```
# Check script permissions
ls -l /path/to/script.sh
# Make it executable
chmod +x /path/to/script.sh
# Check directory permissions
ls -ld /path/to/directory
# Use correct user or permissions
sudo chown user:group /path/to/script.sh
# Use absolute paths in cron
/usr/local/bin/script.sh
```

6. Incident 4: Job Runs Twice and Corrupts an Output File

Possible Causes:

- Overlapping executions (previous run still running).
- Multiple cron entries.
- Script takes longer than schedule interval.
- No locking mechanism.

heck and Fix:

```
# Check if multiple entries exist
crontab -l | grep script.sh

# Use flock to prevent overlapping runs
0 * * * * flock -n /tmp/myjob.lock \
  /usr/local/bin/script.sh >> /var/log/
  /myjob.log 2>&1

# Check running processes
ps -ef | grep script.sh
```

7. Incident 5: Job Appears Successful but Produces No Result

Possible Causes:

- Command exits with code 0 but did not do work.
- Wrong paths or file names.
- Empty input or failed API call.
- Output written to unexpected location.
- Logic error in the script.

Check and Fix:

```
# Check exit code
echo $?
# Verify output files
ls -l /path/to/output
# Add detailed logging
/usr/local/bin/script.sh \
  >> /var/log/myjob.log 2>&1
# Add checks in script
if [ ! -f /path/to/output/file ]; then
  echo "ERROR: Output file not found"
f exit 1
```

8. System Log Locations

Location	Description
/var/log/cron	Cron service logs (some systems)
/var/log/syslog	System logs (includes cron on Debian/Ubuntu)
journalctl -u cron	Cron service logs (systemd)
/var/log/messages	System logs (RHEL/CentOS)
/var/log/myjob.log	Your script execution log

9. Key Takeaways

- Always investigate using real evidence (logs and exit codes).
- Check the correct user, environment and PATH.
- Verify cron service is running.
- Use absolute paths and proper permissions.
- Prevent overlapping runs with locking.
- Log both stdout and stderr.
- Do not assume success – always check the result.
- Fix the root cause, not just the symptom.

Page 20: Build a Production-Aware Cron Automation System

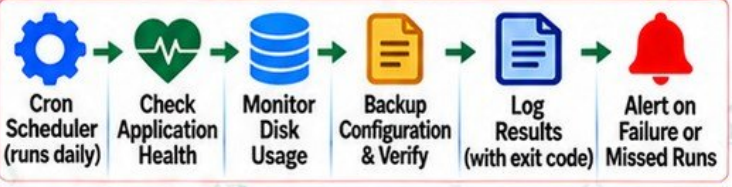
VERIQTA

1. Project Overview

Build a scheduled application maintenance workflow that performs the following:

1. Checks application health.
2. Monitors disk usage.
3. Creates and verifies a configuration backup.
4. Prevents overlapping executions.
5. Records execution results.
6. Alerts on failures or missed runs.

2. Workflow Architecture



3. Cron Schedule

```
0 2 * * * /usr/local/bin/app-maintenance.sh >> /var/log/app-maintenance.log 2>&1
```

Runs daily at 2:00 AM. Adjust the time based on your environment.

4. Main Script: app-maintenance.sh

```
#!/bin/bash
# Production-aware application maintenance
APP_NAME="myapp"
LOG_FILE="/var/log/app-maintenance.log"
LOCK_FILE="/var/lock/app-maintenance.lock"
BACKUP_DIR="/opt/backups"
CONFIG_DIR="/etc/myapp"
ALERT_EMAIL="admin@example.com"
DISK_THRESHOLD=80
MAX_AGE_HOURS=26  # For missed run check

log() {
   echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}
send_alert() {
   local subject="$1"
   local message="$2"
   echo "$message" | mail -s "$subject" "$ALERT_EMAIL"
   log "ALERT: $subject"
}
# Prevent overlapping executions
exec 9>"$LOCK_FILE"
if ! flock -n 9; then
   log "Another instance is already running. Exiting."
   exit 1
fi
# 1. Check application health (example: HTTP endpoint)
log "Checking application health..."
if ! curl -sf http://localhost/health > /dev/null; then
   send_alert "Application Health Check Failed" "App is not responding."
   exit 1
fi
log "Application is healthy."
# 2. Monitor disk usage
log "Checking disk usage..."
USAGE=$(df -h / | awk 'NR==2 {gsub("%","",$5); print $5}')
if [ "$USAGE" -gt "$DISK_THRESHOLD" ]; then
   send_alert "Disk Usage High" "Disk usage is ${USAGE}% (threshold: ${DISK_THRESHOLD}%)."
fi
log "Disk usage: ${USAGE}%"
# 3. Create and verify configuration backup
log "Creating configuration backup..."
BACKUP_FILE="$BACKUP_DIR/myapp-config-$(date '+%Y%m%d-%H%M%S').tar.gz"
tar -czf "$BACKUP_FILE" "$CONFIG_DIR"
if [ $? -ne 0 ] || [ ! -s "$BACKUP_FILE" ]; then
   send_alert "Backup Failed" "Backup file was not created or is empty."
fi exit 1
log "Backup created: $BACKUP_FILE"
log "Maintenance completed successfully."
exit 0
```

5. Missed Run Detection Script (run hourly)

```
#!/bin/bash
LOG_FILE="/var/log/app-maintenance.log"
MAX_AGE_HOURS=26
if [ ! -f "$LOG_FILE" ]; then
   echo "ALERT: Log file not found." | mail -s "Cron Missed Runs" admin@example.com
   exit 1
fi
LAST_RUN=$(stat -c %Y "$LOG_FILE")
NOW=$(date +%s)
AGE_HOURS=$(( (NOW - LAST_RUN) / 3600 ))
if [ "$AGE_HOURS" -gt "$MAX_AGE_HOURS" ]; then
   echo "ALERT: app-maintenance.sh has not run for $AGE_HOURS hours." |
      mail -s "Cron Missed Run" admin@example.com
fi
```

6. Add Missed Run Check to Crontab (hourly)

```
0 * * * * /usr/local/bin/check-missed-run.sh >> /var/log/cron-monitor.log 2>&1
```

7. Example Log Output

```
[2026-10-24 02:00:01] Checking application health...
[2026-10-24 02:00:02] Application is healthy.
[2026-10-24 02:00:02] Checking disk usage...
[2026-10-24 02:00:02] Disk usage: 45%
[2026-10-24 02:00:03] Creating configuration backup...
[2026-10-24 02:00:05] Backup created: /opt/backups/myapp-config-20261024-020005.tar.gz
[2026-10-24 02:00:05] Maintenance completed successfully.
```

8. Production-Readiness Checklist

	Area	Checklist Items
✅	Security	Run with a dedicated service account (no root).
✅	Permissions	Use correct file and directory permissions.
✅	Secrets	Do not store secrets in scripts (use environment variables or a secrets manager).
✅	Testing	Test manually and from cron.
✅	Reliability	Prevent overlapping runs (flock or lock file).
✅	Logging	Log stdout, stderr, timestamps and exit codes.
✅	Monitoring	Alert on failures and missed runs.
✅	Recovery	Verify backups and test restore procedure.
✅	Ownership	Document the job, purpose, schedule and owners.
✅	Review	Regularly review logs, disk usage and job performance.

9. Key Takeaways

- ✅ Combine health checks, monitoring, backups, logging and alerting.
- ✅ Prevent overlapping executions.
- ✅ Detect and alert on missed runs.
- ✅ Use least privilege and secure file permissions.
- ✅ Always verify results instead of assuming success.
- ✅ Test thoroughly before running in production.